

## IFN647 Week 11 Workshop: Text Classification & Clustering

\*\*\*\*\*

### Solutions for Task 1 and Task 2 (a) & (c)

**Task 1.** Understand how to Validate a Model with tunable parameters

#### Solution for (a):

**Step 1: Split the dataset into 5 folds**

Fold 1: D1  
Fold 2: D2  
Fold 3: D3  
Fold 4: D4  
Fold 5: D5

**Step 2: Grid search over  $\lambda$**

For each  $\lambda$ , **Example:  $\lambda = 0.3$** , calculate MAP for using Validation Sets:

| Fold | Train Set   | Validation Set | MAP  |
|------|-------------|----------------|------|
| 1    | D2+D3+D4+D5 | D1             | 0.28 |
| 2    | D1+D3+D4+D5 | D2             | 0.30 |
| 3    | D1+D2+D4+D5 | D3             | 0.27 |
| 4    | D1+D2+D3+D5 | D4             | 0.29 |
| 5    | D1+D2+D3+D4 | D5             | 0.31 |

Then calculate the average MAP, Avg, for the 5 folds:

$$\text{Avg} = \frac{0.28 + 0.30 + 0.27 + 0.29 + 0.31}{5} = 0.29$$

And, repeat for all  $\lambda$  values:

| $\lambda$ | Avg MAP |
|-----------|---------|
| 0.0       | 0.21    |
| 0.1       | 0.25    |
| 0.2       | 0.28    |
| 0.3       | 0.29    |
| 0.4       | 0.31    |
| 0.5       | 0.32    |
| 0.6       | 0.33    |
| 0.7       | 0.30    |

| $\lambda$ | Avg MAP |
|-----------|---------|
| 0.8       | 0.27    |
| 0.9       | 0.23    |
| 1.0       | 0.18    |

**Step 3: Select best  $\lambda$**

$$\lambda^* = 0.6(\text{highest MAP})$$

**Step 4: Final recommendation**

Train on full dataset using  $\lambda = 0.6$

**Solution for (b):**

**(i) Grid search -**

**Step 1: Define parameter ranges**

- $k_1 \in \{1.0, 1.2, 1.5, 2.0\}$
- $b \in \{0.5, 0.75, 1.0\}$

**Step 2: Construct the parameter grid**

All possible combinations:

- (1.0, 0.5), (1.0, 0.75), (1.0, 1.0)
- (1.2, 0.5), (1.2, 0.75), (1.2, 1.0)
- (1.5, 0.5), (1.5, 0.75), (1.5, 1.0)
- (2.0, 0.5), (2.0, 0.75), (2.0, 1.0)

**Step 3: Evaluate each combination**

For each  $(k_1, b)$  pair:

1. Run BM25 on the validation dataset
2. Generate ranked results
3. Compute an effectiveness metric (e.g., MAP or DCG@10)

**Step 4: Select the best parameters**

- Choose the  $(k_1, b)$  pair that achieves the highest evaluation score

### **Pseudocode:**

Algorithm 1

Inputs:

A BM25 evaluate\_model, a validation dataset, parameter ranges K=[1.0, 1.2, 1.5, 2.0] and B=[0.5, 0.75, 1.0]

Method:

```
best_score = -inf (a very small value)
best_params = None

for k1 in K:
    for b in B:
        score = evaluate_model(k1, b)
        if score > best_score:
            best_score = score
            best_params = (k1, b)
# print("Best parameters:", best_params)
return best_params
```

### **(ii) Systematic variation -**

Instead of exhaustively testing all parameter combinations (as in grid search), you vary parameters **step by step** to observe their individual and combined effects.

#### **Step 1: Tune one parameter while fixing others**

Fix  $b = 0.75$ , vary  $k_1$ :

- $k_1 \in \{1.0, 1.2, 1.5, 2.0\}$

Evaluate each setting using a validation metric (e.g., MAP):

| $k_1$ | MAP   |
|-------|-------|
| 1.0   | 0.245 |
| 1.2   | 0.262 |
| 1.5   | 0.271 |
| 2.0   | 0.265 |

Best:  $k_1 = 1.5$

#### **Step 2: Tune the second parameter using the best value**

Fix  $k_1 = 1.5$ , vary  $b$ :

- $b \in \{0.5, 0.75, 1.0\}$

| $b$  | MAP   |
|------|-------|
| 0.5  | 0.268 |
| 0.75 | 0.271 |
| 1.0  | 0.276 |

Best:  $b = 1.0$

### Step 3: (Optional) Refinement

You may further refine around the best values:

- $k_1 \in \{1.4, 1.5, 1.6\}$
- $b \in \{0.9, 1.0\}$

### Pseudocode:

Algorithm 2

Inputs:

A BM25 evaluate\_model, a validation dataset, parameter ranges  $K = [1.0, 1.2, 1.5, 2.0]$  and  $B = [0.5, 0.75, 1.0]$

Method:

```
# Step 1: tune parameter k1
best_k1 = None
best_score = -inf
for k1 in K:
    score = evaluate_model(k1, b=0.75) #0.75 - default value for b
    if score > best_score:
        best_score = score
        best_k1 = k1

# Step 2: tune parameter b
best_b = None
best_score = -inf
for b in K:
    score = evaluate_model(best_k1, b)
    if score > best_score:
        best_score = score
        best_b = b
# print("Best parameters:", best_k1, best_b)
return (best_k1, best_b)
```

## Task 2. Linear and non-linear SVM classifiers

### Solution for (a):

```
x_train = [[0, 0, 0, 0, 2], [3, 0, 1, 0, 1], [0, 0, 0, 0, 1], [2, 0, 3, 0, 2], [5, 2, 0, 0, 1], [0, 0, 1, 0, 1], [0, 1, 1, 0, 1], [0, 0, 0, 0, 1], [0, 0, 0, 0, 1], [1, 1, 0, 1, 2]]
```

```
y_train = [-1, 1, -1, 1, 1, -1, -1, -1, -1, -1]
```

### Solution for (c)

The Confusion Matrix for SVC:

```
[[6 0]
 [1 3]]
```

The Confusion Matrix for LinearSVC:

```
[[6 0]
 [1 3]]
```

TP = 6

FP = 0

FN = 1

TN = 3

Accuracy = (6+3) / (6+0+1+3) = 90%